# LAB 7

Fuzz Testing for Input Validation Bugs

## Vending Machine Application

This report documents the use of Hypothesis property-based fuzz testing to identify input-validation failures in a Python vending machine program, the defensive changes made to the functions, and the final successful test run.

| | |
|---|---|
| **Git repository** | https://github.com/ashlesh-rv/SE-LABS-PES1UG24CS088.git |
| **Folder** | Lab 7 - Fuzz Testing |
| **Application file** | vending_machine.py |
| **Test file** | test_vending_machine.py |
| **Environment shown** | Python 3.12.7, pytest 9.2.0, Hypothesis 6.170.15 |
| **Test command** | python -m pytest test_vending_machine.py -v -p no:cacheprovider |

### Objective

To explore fuzz testing as a security and quality-assurance technique by identifying and fixing bugs in the assigned Python script using the Hypothesis fuzzing library.

### Submission contents

| | |
|---|---|
| **1. Consolidated PDF** | Screenshots of the original run, failing fuzz tests, corrected application, final successful tests, and complete source listings. |
| **2. Python application** | Fixed vending_machine.py (the assigned scenario's application filename). |
| **3. Hypothesis tests** | Completed test_vending_machine.py with fuzz tests for all three functions. |

# 1. Fuzz Testing Method

Hypothesis is a property-based testing library. Instead of writing only a few fixed examples, the tests describe a property - each function should not crash - and Hypothesis generates a variety of inputs to try to find counterexamples. The strategy used for all three tests is st.text() | st.none(), which includes arbitrary text and None.

The test file contains one test for each target function: parse_coins(), slot_position(), and change_due().

# 2. Bugs Discovered

| Function | Hypothesis example | Observed problem |
|---|---|---|
| parse_coins() | Empty string; None | ValueError converting an empty token to int; AttributeError because None has no split(). |
| slot_position() | Empty string; '0'; None | IndexError for missing first character; ValueError for an invalid number; TypeError for None. |
| change_due() | '/' ; empty string; None | ValueError for missing or invalid payment parts; AttributeError for None. |

The original run collected all three tests and marked all three as FAILED. Hypothesis provided small failing inputs that revealed assumptions in the original functions: every input was treated as a correctly formatted string.

# 3. Fixes Applied

**parse_coins():** Validates the input type and non-empty content before splitting. Integer conversion is protected with try/except. Negative coin values are rejected with an empty-list result.

**slot_position():** Validates the input type and basic slot format before indexing or converting the column. Invalid codes return None instead of raising an exception.

**change_due():** Validates the input type and slash-separated payment format. Numeric conversion is protected, and negative amounts or underpayment return a safe zero-change result.

For invalid inputs, the functions now return safe values rather than allowing an exception to escape. Valid example inputs continue to produce the expected output.

# 4. Fuzz Test Evidence - Before Fixes

Command: `python -m pytest test_vending_machine.py -v -p no:cacheprovider`

The first successful Hypothesis execution collected three tests and reported failures in all three target functions.

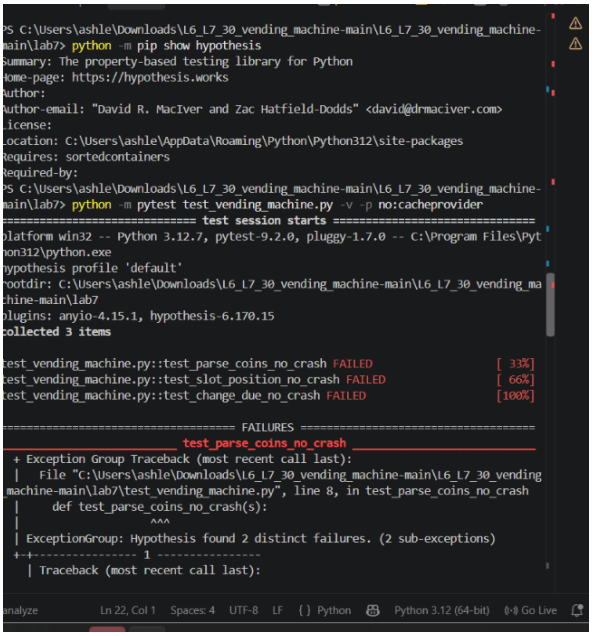



*Figure 1. Pytest summary showing all three Hypothesis tests failed.*

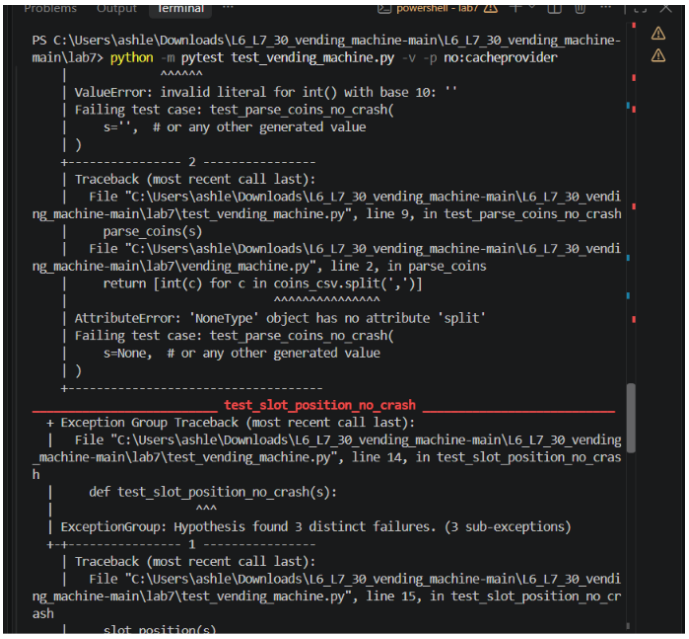



*Figure 2. Detailed failing examples and exceptions reported by Hypothesis.*

# 5. Corrected Application Execution

The fixed application was executed with the handout's example inputs:

| Input prompt | Entered value | Output |
|---|---|---|
| Coin values | 5,10,25 | [5, 10, 25] |
| Slot code | B7 | (1, 6) |
| Paid/price | 200/150 | 10 five-coins |

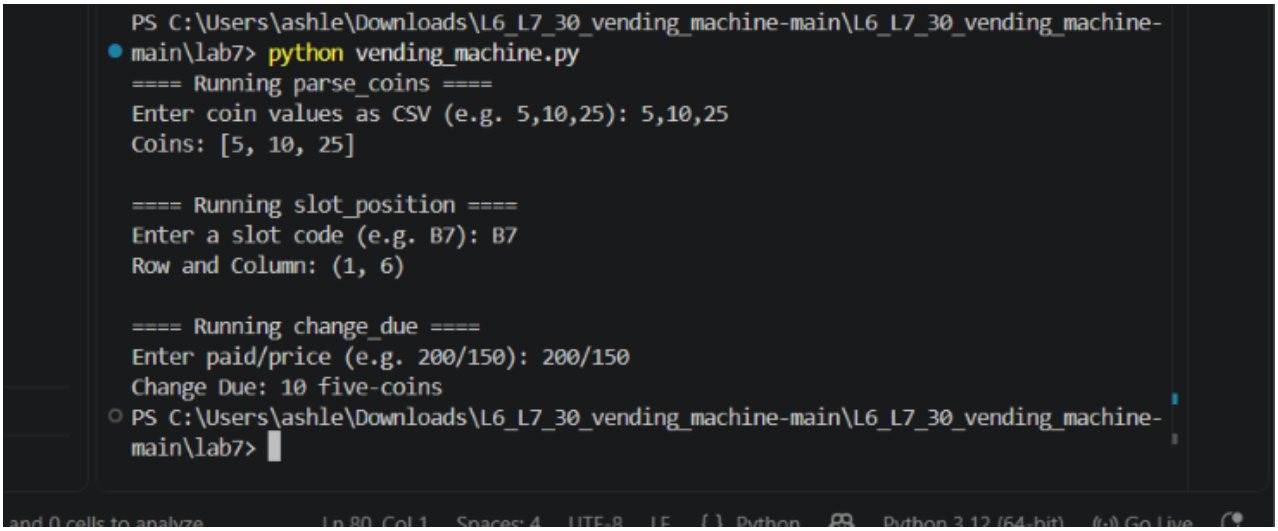



*Figure 3. Corrected vending machine application running with valid sample inputs.*

# 6. Final Fuzz Test Execution

The same test file was executed again after the fixes. All three tests passed.

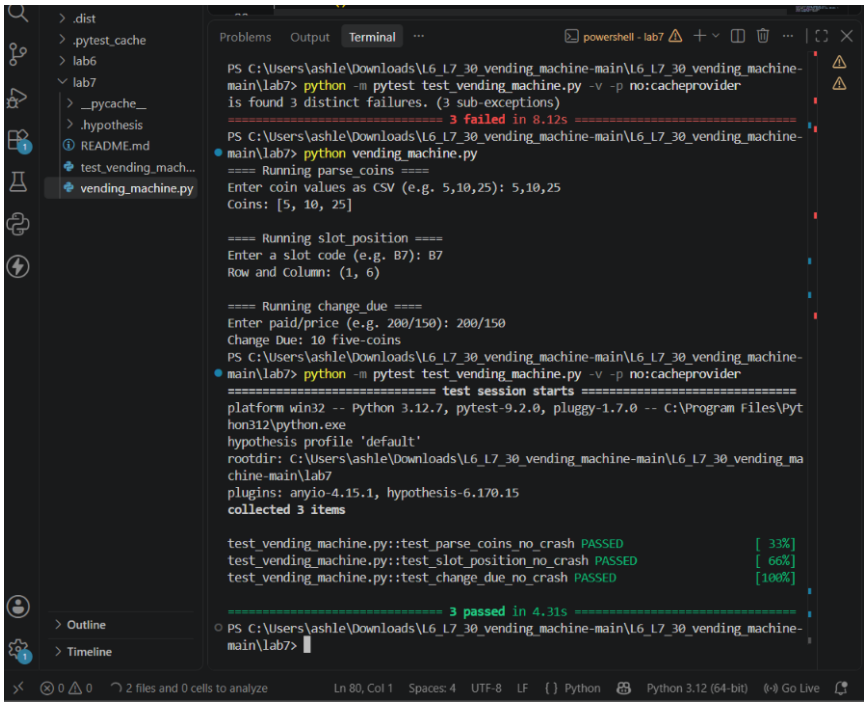



*Figure 4. Final pytest output showing 3 passed.*

# 7. Python Source Code - vending_machine.py

Complete corrected application source code. This is the actual module name used by the assigned vending-machine scenario and imported by test_vending_machine.py.

```
def parse_coins(coins_csv):
    if not isinstance(coins_csv, str) or not coins_csv.strip():
        return []

    try:
        coins = [int(c.strip()) for c in coins_csv.split(',')]

        if any(c < 0 for c in coins):
            return []

        return coins
    except ValueError:
        return []


def slot_position(code):
    if not isinstance(code, str):
        return None

    code = code.strip()

    if len(code) < 2:
        return None

    if not code[0].isalpha() or not code[1:].isdigit():
        return None

    row = ord(code[0].upper()) - ord('A')
    column = int(code[1:]) - 1

    if column < 0:
        return None

    return row, column


def change_due(payment):
    if not isinstance(payment, str):
        return "0 five-coins"

    parts = payment.split('/')

    if len(parts) != 2:
        return "0 five-coins"

    try:
        paid = int(parts[0].strip())
        price = int(parts[1].strip())

        if paid < 0 or price < 0 or paid < price:
            return "0 five-coins"

        change = paid - price
        return str(change // 5) + ' five-coins'

    except ValueError:
        return "0 five-coins"


def main():
    print("==== Running parse_coins ====")
    user_input = input("Enter coin values as CSV (e.g. 5,10,25): ")
    result = parse_coins(user_input)
    print("Coins:", result)

    print("\n==== Running slot_position ====")
    user_input = input("Enter a slot code (e.g. B7): ")
    result = slot_position(user_input)
    print("Row and Column:", result)

    print("\n==== Running change_due ====")
    user_input = input("Enter paid/price (e.g. 200/150): ")
    result = change_due(user_input)
    print("Change Due:", result)


if __name__ == "__main__":
    main()
```

# 8. Python Source Code - test_vending_machine.py

Complete Hypothesis-based fuzz test source code for all three application functions.

```
from hypothesis import given, strategies as st
from vending_machine import parse_coins, slot_position, change_due


# Test 1: Fuzz testing for parse_coins()
@given(st.text() | st.none())
def test_parse_coins_no_crash(s):
    parse_coins(s)


# Test 2: Fuzz testing for slot_position()
@given(st.text() | st.none())
def test_slot_position_no_crash(s):
    slot_position(s)


# Test 3: Fuzz testing for change_due()
@given(st.text() | st.none())
def test_change_due_no_crash(s):
    change_due(s)
```

## Test command used

```
python -m pytest test_vending_machine.py -v -p no:cacheprovider
```

Expected final result recorded in the evidence: 3 passed.

# 9. GitHub Repository Evidence

The Lab 7 folder was pushed to the user's main Software Engineering Labs repository. The screenshot below shows the three files committed in the folder.

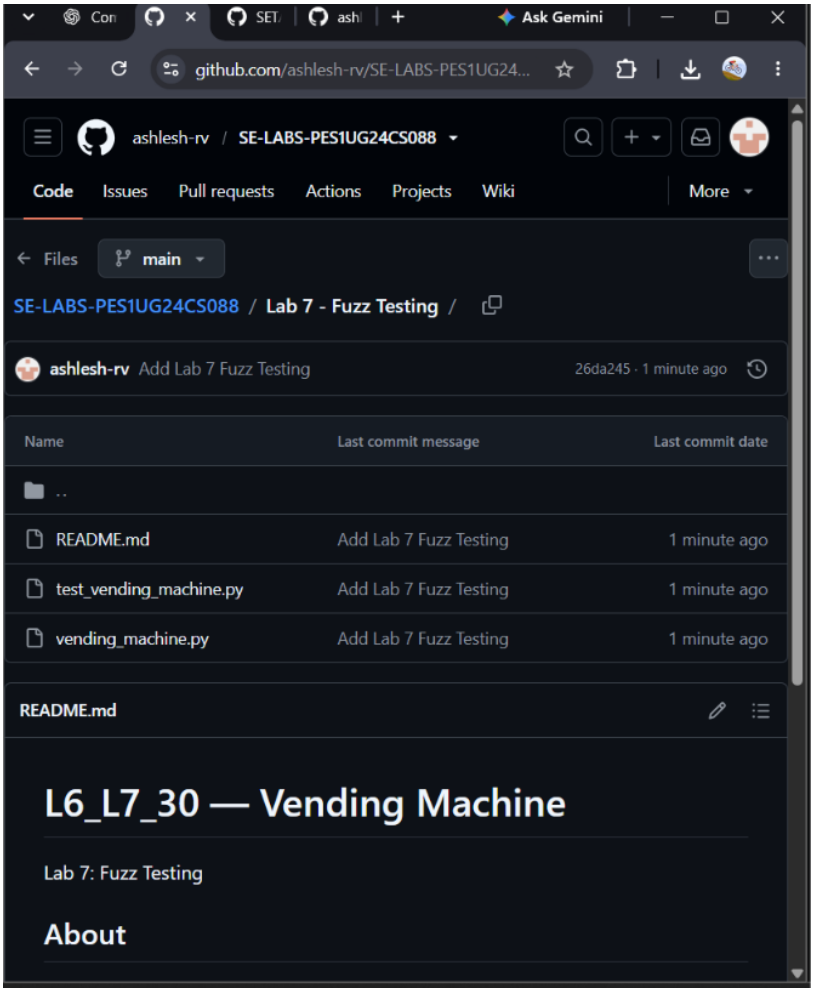



*Figure 5. GitHub folder containing README.md, test_vending_machine.py, and vending_machine.py.*

**Repository:** https://github.com/ashlesh-rv/SE-LABS-PES1UG24CS088.git

# 10. Conclusion

Hypothesis fuzz testing identified unexpected-input failures in all three vending machine functions. Input checks and defensive error handling were added. The corrected application produced the expected output for valid sample inputs, and the final Hypothesis run passed all three tests.

## Final submission checklist

| Required item | Current evidence |
|---|---|
| Original application execution screenshot | **Pending - not supplied.** It is not included because no screenshot of the original application run was provided in this conversation. Do not present the failing pytest screenshot as this evidence. |
| Failing fuzz-test screenshot | Included - Figures 1 and 2. |
| Corrected application execution screenshot | Included - Figure 3. |
| Final successful test screenshot | Included - Figure 4 (3 passed). |
| Both Python source-code listings | Included in full - Sections 7 and 8. |
| GitHub folder screenshot | Included - Figure 5. |